Student Name: ShadowBoxing

Assignment:

Notes:

Project Name: SetAuton

Project Type: C++

Date: Tue Nov 03 2026

```
#pragma region VEXcode Generated Robot Configuration
// Make sure all required headers are included.
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <math.h>
#include <string.h>


#include "vex.h"

using namespace vex;

// Brain should be defined by default
brain Brain;


// START V5 MACROS
#define waitUntil(condition)                                                   \
  do {                                                                         \
    wait(5, msec);                                                             \
  } while (!(condition))

#define repeat(iterations)                                                     \
  for (int iterator = 0; iterator < iterations; iterator++)
// END V5 MACROS


// Robot configuration code.



// generating and setting random seed
void initializeRandomSeed(){
  int systemTime = Brain.Timer.systemHighResolution();
  double batteryCurrent = Brain.Battery.current();
  double batteryVoltage = Brain.Battery.voltage(voltageUnits::mV);

  // Combine these values into a single integer
  int seed = int(batteryVoltage + batteryCurrent * 100) + systemTime;

  // Set the seed
  srand(seed);
}



void vexcodeInit() {

  //Initializing random seed.
  initializeRandomSeed();
}


// Helper to make playing sounds from the V5 in VEXcode easier and
// keeps the code cleaner by making it clear what is happening.
void playVexcodeSound(const char *soundName) {
  printf("VEXPlaySound:%s\n", soundName);
  wait(5, msec);
}

#pragma endregion VEXcode Generated Robot Configuration
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
```

```cpp
#include "vex.h"

using namespace vex;

// sdcard SDCard = Brain.Sdcard();
controller Controller1;

// Motors (adjust ports as needed)
motor Right_Top( PORT18, gearSetting::ratio6_1, false);
motor Right_Middle(PORT19, gearSetting::ratio6_1, true);
motor Right_Bottom(PORT20, gearSetting::ratio6_1, false);

motor Left_Top(PORT8, gearSetting::ratio6_1, true);
motor Left_Middle(PORT9, gearSetting::ratio6_1, false);
motor Left_Bottom(PORT10, gearSetting::ratio6_1, true);

// Motor groups
motor_group LeftDrive(Left_Top,Left_Middle,Left_Bottom);
motor_group RightDrive(Right_Top, Right_Middle,Right_Bottom);

// elevator motors
motor elevatorT(PORT5, gearSetting::ratio18_1, false );
motor elevatorB(PORT4, gearSetting::ratio18_1, true );
//intake motors
motor tintakeL(PORT2, gearSetting::ratio18_1, true );
motor tintakeR(PORT12, gearSetting::ratio18_1, false );

motor bintakeL(PORT1, gearSetting::ratio18_1, true );
motor bintakeR(PORT11, gearSetting::ratio18_1, false );

// motor groups
motor_group Elevator( elevatorT, elevatorB );

motor_group Tintake( tintakeL, tintakeR );
motor_group Bintake( bintakeL, bintakeR );



void update(int x){
  // Begin project code

  // Timing variables
  int timeOld = Brain.Timer.time(msec);
  int timeNew = 0;
  int deltaTime = 0;

  // Clear file at start
  FILE* file = fopen("/usd/rerun.txt", "w");

  if(file != NULL) {
    fprintf(file, "Hello world\n");
    fclose(file);
  } else {
    Brain.Screen.print("FAIL");
  }

  while (true) {

    // Controller input
    int dir = Controller1.Axis3.position(); // forward/back
    int turn = Controller1.Axis1.position(); // turning

    // Drive
    LeftDrive.spin(forward, dir + turn, percent);
    RightDrive.spin(forward, dir - turn, percent);
```

```
131       // intakes
132       Elevator.spin(forward, 100, percent);
133       Tintake.spin(forward, 100, percent);
134       Bintake.spin(forward, 100, percent);
135
136
137       // Small delay (loop timing)
138       wait(20, msec);
139
140       auto start = std::chrono::steady_clock::now();
141
142       // Get motor velocities
143
144       int leftSpeed = LeftDrive.velocity(percent);
145       int rightSpeed = RightDrive.velocity(percent);
146       // intakes
147       int ElevatorSpeed = Elevator.velocity(percent);
148       int TintakeSpeed = Tintake.velocity(percent);
149       int BintakeSpeed = Bintake.velocity(percent);
150
151
152       // Open file (append)
153       FILE* file = fopen("/usd/rerun.txt", "a");
154
155       // Write motor commands
156
157       fprintf(file, "left_mg.spin(forward, %d, percent);\n", leftSpeed);
158       fprintf(file, "right_mg.spin(forward, %d, percent);\n", rightSpeed);
159       // intakes
160       fprintf(file, "Elevator.spin(forward, %d, percent);\n", ElevatorSpeed);
161       fprintf(file, "Tintake.spin(forward, %d, percent);\n", TintakeSpeed);
162       fprintf(file, "Bintake.spin(forward, %d, percent);\n", BintakeSpeed);
163
164
165       auto current = std::chrono::steady_clock::now();
166       if(std::chrono::duration_cast<std::chrono::seconds>(current - start).count() >
= 60) {
167          fprintf(file, "\n");
168          fprintf(file, "\n");
169          fprintf(file, "\n");
170       }
171       // Timing
172       timeNew = Brain.Timer.time(msec);
173       deltaTime = timeNew - timeOld;
174       timeOld = timeNew;
175
176       // Write delay
177       fprintf(file, "wait(%d, msec);\n", deltaTime);
178
179       fclose(file);
180
181       wait(x, msec);
182     }
183   }
184
185   int main() {
186     update(100);
187   }
188
189
```